

COS 350 Lecture 18

unix: ps

shells

bash

exec

fork

wait

NAME

ps - report a snapshot of the current processes.

SYNOPSIS

ps [options]

DESCRIPTION

ps displays information about a selection of the active processes.

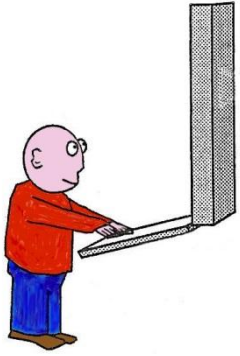
By default, **ps** selects all processes with the same effective user ID as the current user and associated with the same terminal as the invoker.

This version of **ps** accepts several kinds of options:

- 1 UNIX options, may be grouped and must be preceded by a dash.
- 2 BSD options, may be grouped and must not be used with a dash.
- 3 GNU long options, which are preceded by two dashes.

Commonly used options

- f** Full format listing
- l** Long listing (similar shows process state)
- e** Every process on the system
- u username** All process by that user (all terminals)



ps

ps -f

ps -l

ps -ef

Chapter 8. How to write a shell (command interpreter)

linux standard shell is **bash**

- lets you type commands
- finds and runs the programs for these commands
- manages input and output: tty, redirection, & pipes
- has a scripting language

other shells:

- sh - the earliest shell called the “Bourne shell” after its author
- csh, tcsh, ksh, dash

shells differ in their special features

- history
- command completion
- scripting language

bash – name is a pun “Bourne again shell”

- has scripting language of original Bourne shell
- added feature of other shells like history and command completion

The shell loop:

1. get a command
2. create a new process `fork()`
3. the new process runs the command `exec()`
4. original process waits for the command to finish `wait()`
5. repeat until exit the shell

NAME

execvp - execute a file

SYNOPSIS

```
#include <unistd.h>
```

```
int execvp(const char *file, char *const argv[]);
```

DESCRIPTION

The **exec()** family of functions replaces the current process image with a new process image.

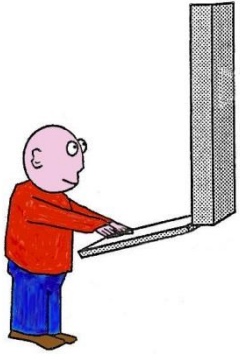
file is the pathname of a file which is to be executed

argv[] is an array of pointers to strings that represent the argument list available to the new program. The first argument, by convention, should point to the filename associated with the file being executed. The array of pointers must be terminated by a NULL pointer.

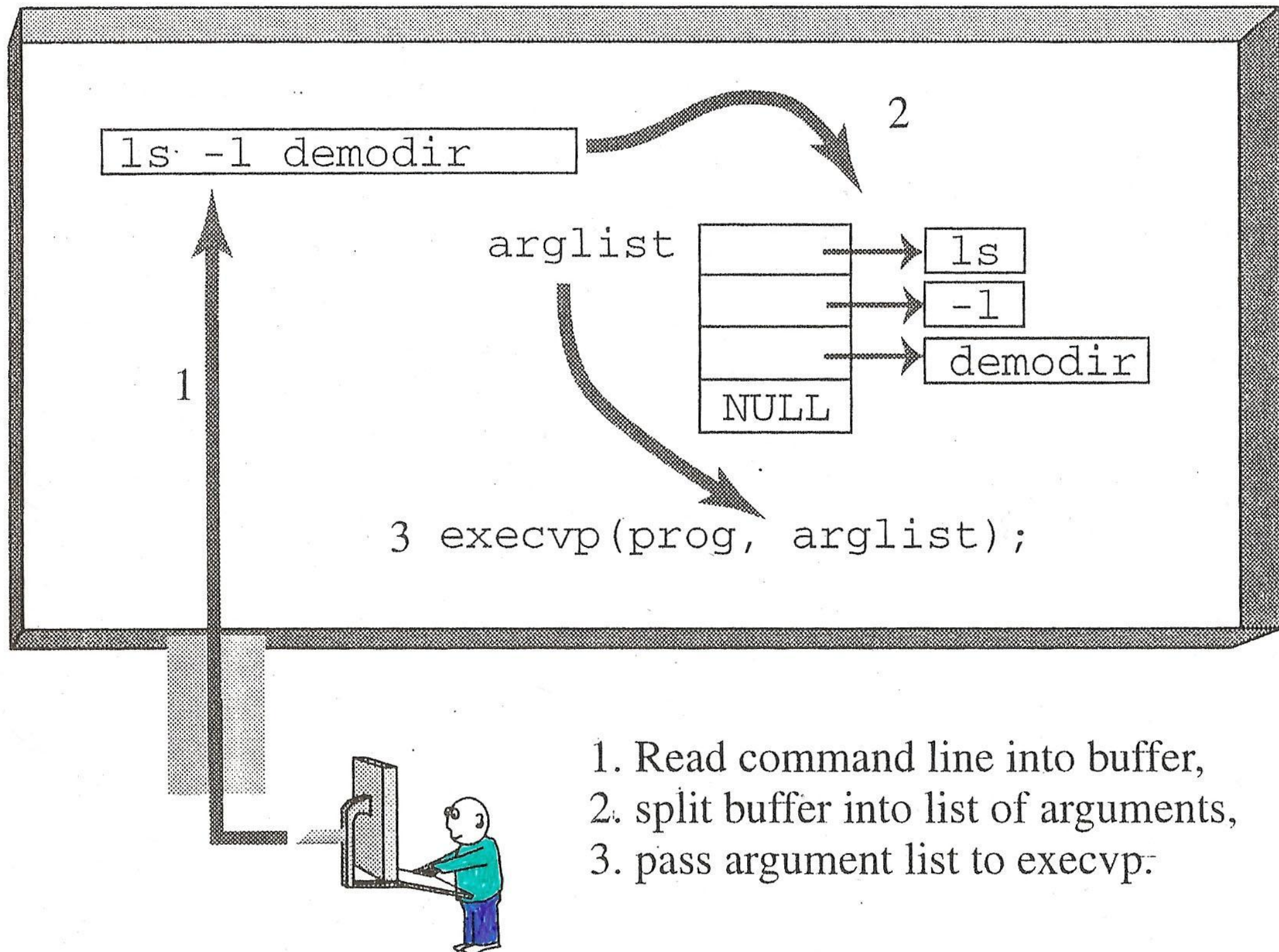
RETURN VALUE

Normally this does not return.

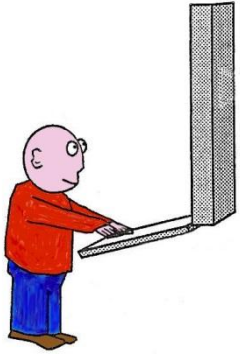
On error it returns -1 and the global variable **errno** will be set to indicate the error.



exec1.c

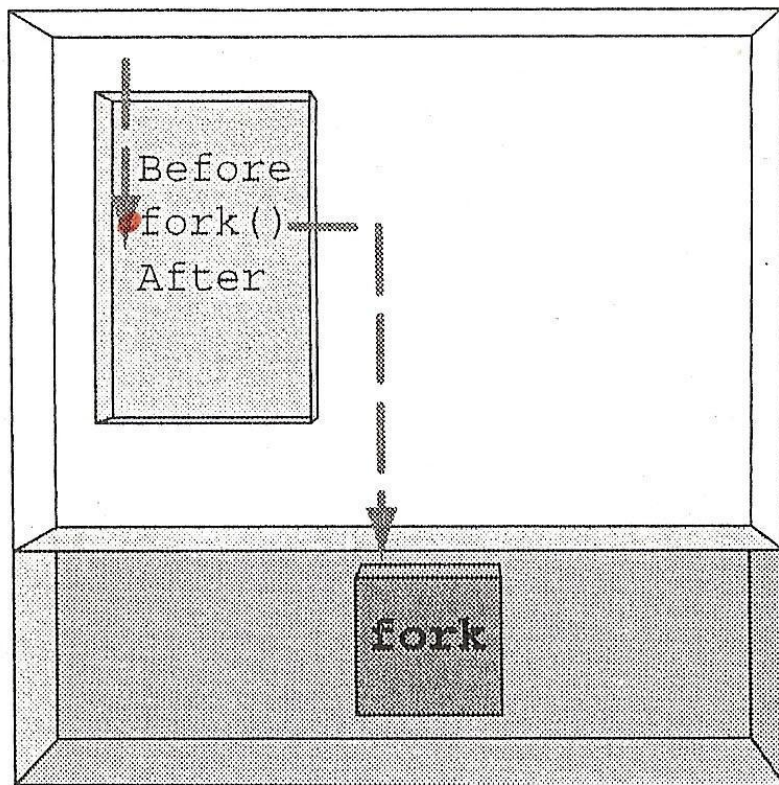


1. Read command line into buffer,
2. split buffer into list of arguments,
3. pass argument list to `execvp`.

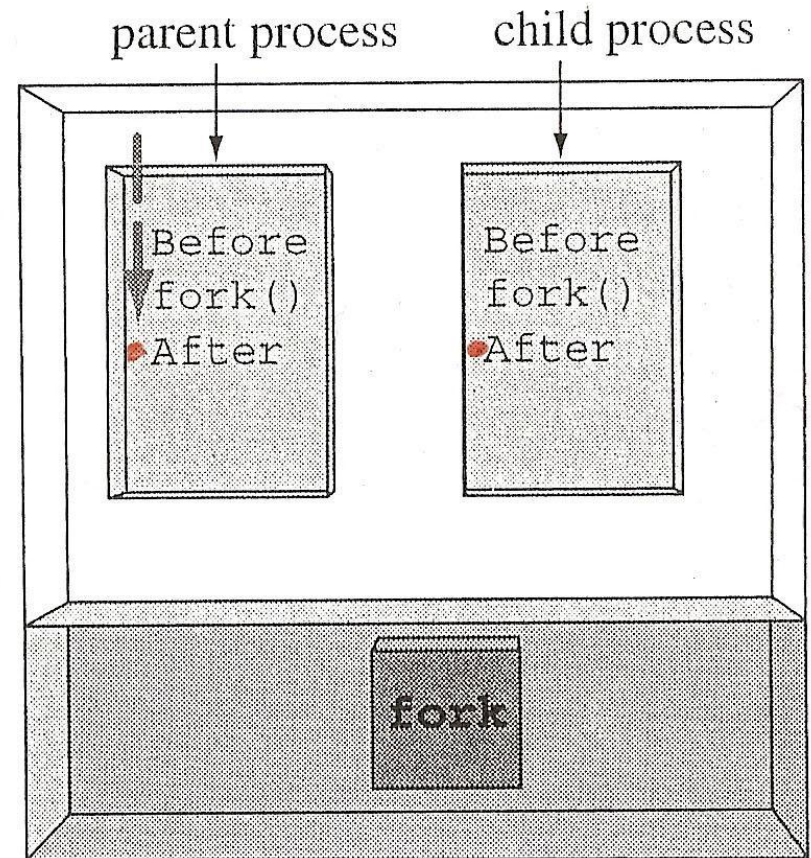


psh1.c

Before fork:



After fork:



The new process contains the same code and data as the parent process.

FIGURE 8.8

`fork()` makes a copy of a process.

NAME

fork - create a child process

SYNOPSIS

```
#include <unistd.h>
```

```
pid_t fork(void);
```

DESCRIPTION

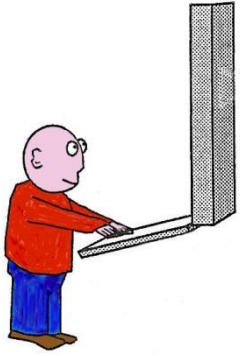
fork() creates a new process by duplicating the calling process. The new process, referred to as the child, is an exact duplicate of the calling process, referred to as the parent, except:

- * The child has its own unique process ID.
- * The child's parent process ID equals the parent's process ID.

RETURN VALUE

On success: The PID of the child process is returned in the parent, and 0 is returned in the child.

On failure: -1 is returned in the parent.



forkdemo1.c
forkdemo2.c

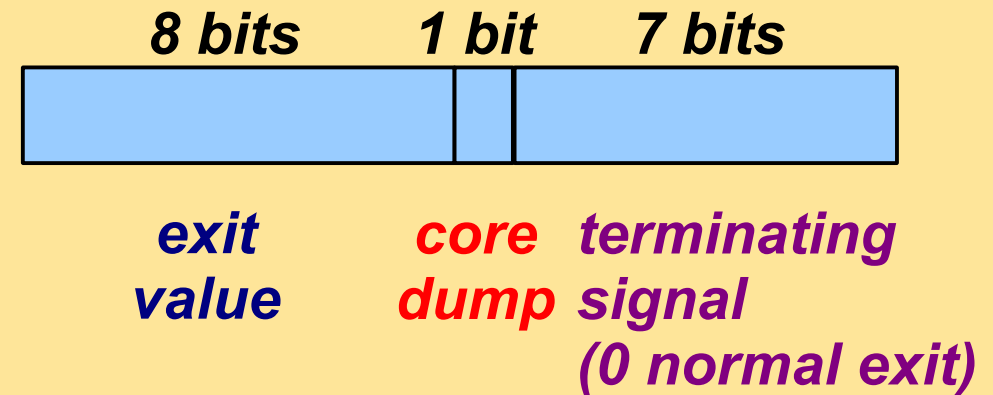
NAME

wait - wait for process termination

SYNOPSIS

```
#include <sys/types.h>
#include <sys/wait.h>
```

```
pid_t wait(int *status);
```



DESCRIPTION

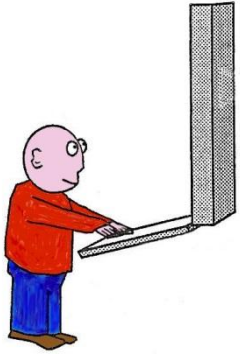
The `wait()` system call suspends execution of the calling process until one of its children terminates.

RETURN VALUE

On success: returns the process ID of the terminated child.
On error: -1 is returned.

SEE ALSO

`waitpid(pid, status, options)`



waitdemo.c

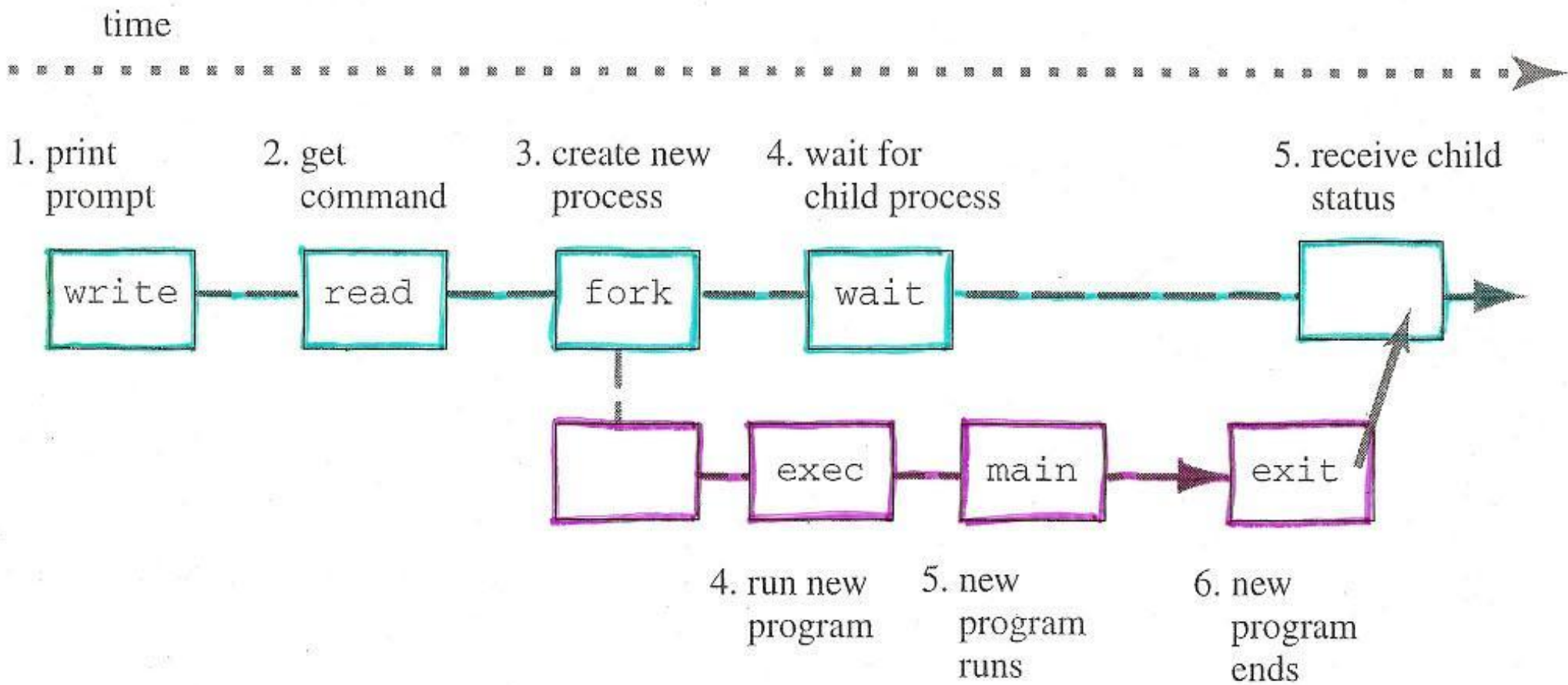


FIGURE 8.13

Shell loop with `fork()`, `exec()`, and `wait()`.